ADA Lab Journal

Analysis and Design of Algorithms

Simplified Code — All Labs

LAB 1 — Minimum Cost Spanning Tree using Kruskal's Algorithm

Sort all edges by weight. Pick smallest edge that does NOT form a cycle until (V-1) edges are selected. Use Union-Find to detect cycles.

```
#include <stdio.h>
#include <stdlib.h>

struct Edge { int src, dest, weight; };
int parent[100];

int find(int i) {
    if (parent[i] != i)
        parent[i] = find(parent[i]);
    return parent[i];
}

void unite(int x, int y) {
    parent[find(x)] = find(y);
}

int compare(const void *a, const void *b) {
    return ((struct Edge*)a)->weight - ((struct Edge*)b)->weight;
}

int main() {
    int V, E;
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter number of edges: ");
    scanf("%d", &E);

    struct Edge edges[E];
    printf("Enter source destination and weight of each edge:\n");
    for (int i = 0; i < E; i++)
        scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight)

    qsort(edges, E, sizeof(edges[0]), compare);
    for (int i = 0; i < V; i++) parent[i] = i;

    int minCost = 0;
    printf("\nEdges in Minimum Cost Spanning Tree:\n");
    for (int i = 0; i < E; i++) {
        int u = edges[i].src, v = edges[i].dest;
        if (find(u) != find(v)) {
            printf("%d -- %d == %d\n", u, v, edges[i].weight);
            minCost += edges[i].weight;
            unite(u, v);
```

```
        }
    }
    printf("\nMinimum Cost of Spanning Tree = %d\n", minCost);
    return 0;
}
```

Sample Output:

```
Edges in Minimum Cost Spanning Tree:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10

Minimum Cost of Spanning Tree = 19
```

LAB 2 — Minimum Cost Spanning Tree using Prim's Algorithm

Start from vertex 0. At each step pick the cheapest edge connecting a visited vertex to an unvisited one. Input: cost adjacency matrix (0 = no edge).

```
#include <stdio.h>
#define INF 99999

int main() {
    int n, cost[100][100], visited[100] = {0};
    int mincost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter the cost adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0) cost[i][j] = INF;
        }

    visited[0] = 1;
    printf("\nEdges in Minimum Spanning Tree:\n");

    for (int edge = 1; edge < n; edge++) {
        int min = INF, a, b;
        for (int i = 0; i < n; i++)
            if (visited[i])
                for (int j = 0; j < n; j++)
                    if (!visited[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        a = i; b = j;
                    }
        printf("%d --> %d = %d\n", a, b, min);
        mincost += min;
        visited[b] = 1;
    }

    printf("\nMinimum Cost = %d\n", mincost);
    return 0;
}
```

Sample Output:

```
Edges in Minimum Spanning Tree:
0 --> 3 = 5
3 --> 2 = 4
0 --> 1 = 10
Minimum Cost = 19
```

LAB 3a — All-Pairs Shortest Path using Floyd's Algorithm

For each intermediate vertex k, check if going through k gives a shorter path. Three nested loops. Time: $O(n^3)$.

```
#include <stdio.h>
#define INF 99999

int main() {
    int n, dist[10][10];

    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix (use %d for infinity):\n", INF);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &dist[i][j]);

    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    printf("\nShortest path matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            dist[i][j] == INF ? printf("INF ") : printf("%d ", dist[i][j]);
        printf("\n");
    }
    return 0;
}
```

Sample Output:

```
Shortest path matrix:
0 3 5 6
5 0 2 3
3 6 0 1
2 5 7 0
```

LAB 3b — Transitive Closure using Warshall's Algorithm

Find if ANY path exists from i to j (reachability). Result is a 0/1 matrix — 1 means reachable.

```
#include <stdio.h>

int main() {
    int n, g[10][10];

    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
```

```
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &g[i][j]);

    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                g[i][j] = g[i][j] || (g[i][k] && g[k][j]);

    printf("\nTransitive closure of the graph:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", g[i][j]);
        printf("\n");
    }
    return 0;
}
```

Sample Output:

```
Transitive closure of the graph:
0 1 1 1
0 0 1 1
0 0 0 1
0 0 0 0
```

LAB 4 — Single Source Shortest Path using Dijkstra's Algorithm

Pick unvisited vertex with minimum distance, mark visited, update its neighbors. Input: adjacency matrix (0 = no edge).

```
#include <stdio.h>
#define INF 99999

int main() {
    int n, cost[100][100], dist[100], visited[100] = {0}, src;

    printf("Enter number of vertices: "); scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (i != j && cost[i][j] == 0) cost[i][j] = INF;
        }

    printf("Enter source vertex: "); scanf("%d", &src);

    for (int i = 0; i < n; i++) dist[i] = cost[src][i];
    dist[src] = 0;
    visited[src] = 1;

    for (int c = 1; c < n - 1; c++) {
        int min = INF, u;
        for (int i = 0; i < n; i++)
            if (!visited[i] && dist[i] < min) { min = dist[i]; u = i; }

        visited[u] = 1;

        for (int i = 0; i < n; i++)
            if (!visited[i] && min + cost[u][i] < dist[i])
                dist[i] = min + cost[u][i];
    }

    printf("\nShortest distances from vertex %d:\n", src);
    for (int i = 0; i < n; i++)
        if (i != src) printf("%d --> %d = %d\n", src, i, dist[i]);
    return 0;
}
```

Sample Output:

```
Shortest distances from vertex 0:
0 --> 1 = 2
0 --> 2 = 5
0 --> 3 = 1
```

LAB 5 — Topological Ordering of a Directed Graph

Kahn's algorithm: enqueue all vertices with indegree 0, process them, reduce neighbors' indegree. If count != n, cycle exists.

```
#include <stdio.h>

int n, adj[100][100], indeg[100];

void topologicalSort() {
    int queue[100], front = 0, rear = 0, count = 0;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (adj[i][j]) indeg[j]++;

    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) queue[rear++] = i;

    printf("Topological Order: ");
    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);
        count++;
        for (int i = 0; i < n; i++)
            if (adj[v][i] && --indeg[i] == 0)
                queue[rear++] = i;
    }
    if (count != n)
        printf("\nGraph has a cycle. Topological sorting not possible.\n");
}

int main() {
    printf("Enter number of vertices: "); scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    topologicalSort();
    return 0;
}
```

Sample Output:

```
Topological Order: 0 1 2 3
```

LAB 6 — 0/1 Knapsack using Dynamic Programming

Each item is taken fully or not at all. knap[i][w] = max value using first i items with capacity w.

```
#include <stdio.h>

int max(int a, int b) { return a > b ? a : b; }

int knapsack(int W, int wt[], int val[], int n) {
    int knap[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                knap[i][w] = 0;
            else if (wt[i-1] <= w)
                knap[i][w] = max(val[i-1] + knap[i-1][w-wt[i-1]],
                                 knap[i-1][w]);
            else
                knap[i][w] = knap[i-1][w];
        }
    }
    return knap[n][W];
}

int main() {
    int val[] = {20, 25, 40};
    int wt[]  = {25, 20, 30};
    int W = 50, n = 3;
    printf("The solution is : %d", knapsack(W, wt, val, n));
    return 0;
}
```

Sample Output:

```
The solution is : 65
```

LAB 7 — Discrete & Continuous Knapsack using Greedy Method

Pick items with highest value/weight ratio first. Take a fraction of last item if it doesn't fit fully (continuous knapsack).

```
#include <stdio.h>

int main() {
    int n = 5;
    int p[] = {3, 3, 2, 5, 1};
    int w[] = {10, 15, 10, 12, 8};
    int W = 10, used[10] = {0};
    int cur_w = W;
    float tot_v = 0;

    while (cur_w > 0) {
        int maxi = -1;
        for (int i = 0; i < n; i++)
            if ((used[i] == 0) && ((maxi == -1) ||
                ((float)w[i]/p[i] > (float)w[maxi]/p[maxi])))
                maxi = i;

        used[maxi] = 1;
        cur_w -= p[maxi];
        tot_v += w[maxi];

        if (cur_w >= 0)
            printf("Added object %d (%d, %d) completely in the bag. Space
                   maxi+1, w[maxi], p[maxi], cur_w);
        else {
            printf("Added %d%% (%d, %d) of object %d in the bag.\n",
                   (int)((1 + (float)cur_w/p[maxi]) * 100), w[maxi], p[max
            tot_v -= w[maxi];
            tot_v += (1 + (float)cur_w/p[maxi]) * w[maxi];
        }
    }
    printf("Filled the bag with objects worth %.2f.\n", tot_v);
    return 0;
}
```

Sample Output:

```
Added object 5 (8, 1) completely in the bag. Space left: 9.
Added object 2 (15, 3) completely in the bag. Space left: 6.
Added object 3 (10, 2) completely in the bag. Space left: 4.
Filled the bag with objects worth 33.00.
```

LAB 8 — Subset Sum Problem (Backtracking)

Find all subsets that sum to d. Include element at k and recurse, then exclude and recurse. Pruning avoids unnecessary branches.

```
#include <stdio.h>
#define MAX 10

int s[MAX], x[MAX], d;

void sumofsub(int p, int k, int r) {
    x[k] = 1;
    if ((p + s[k]) == d) {
        for (int i = 1; i <= k; i++)
            if (x[i]) printf("%d ", s[i]);
        printf("\n");
    }
    else if (p + s[k] + s[k+1] <= d)
        sumofsub(p + s[k], k+1, r - s[k]);

    if ((p + r - s[k] >= d) && (p + s[k+1] <= d)) {
        x[k] = 0;
        sumofsub(p, k+1, r - s[k]);
    }
}

int main() {
    int n, sum = 0;
    printf("\nEnter the n value: "); scanf("%d", &n);
    printf("\nEnter the set in increasing order:\n");
    for (int i = 1; i <= n; i++) { scanf("%d", &s[i]); }
    printf("\nEnter the max subset value: "); scanf("%d", &d);
    for (int i = 1; i <= n; i++) sum += s[i];

    if (sum < d || s[1] > d)
        printf("\nNo subset possible");
    else
        sumofsub(0, 1, sum);
    return 0;
}
```

Sample Output:

```
Enter the n value: 6
Enter the set in increasing order:
1 2 5 6 8 10
Enter the max subset value: 11
1 2 8
5 6
```

LAB 9 — Selection Sort with Timing

Find minimum from unsorted portion and swap to front. O(n²). Measure execution time for different input sizes.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace std::chrono;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int min_idx = i;
        for (int j = i+1; j < n; j++)
            if (arr[j] < arr[min_idx]) min_idx = j;
        int temp = arr[i]; arr[i] = arr[min_idx]; arr[min_idx] = temp;
    }
}

int main() {
    int sizes[] = {5000, 10000, 15000, 20000, 25000};
    srand(time(0));

    for (int s = 0; s < 5; s++) {
        int n = sizes[s];
        int *arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = rand() % 100000;

        auto start = high_resolution_clock::now();
        selectionSort(arr, n);
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<milliseconds>(stop - start);

        cout << "Size: " << n << " Time: " << duration.count() << " ms" <<
        delete[] arr;
    }
    return 0;
}
```

Sample Output:

```
Size: 5000 Time: 55 ms
Size: 10000 Time: 210 ms
Size: 15000 Time: 470 ms
Size: 20000 Time: 820 ms
Size: 25000 Time: 1300 ms
```

LAB 10 — Quick Sort with Timing

Pick pivot, partition into smaller/larger halves, recursively sort each. Average O(n log n). Measure time for n > 5000.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void quicksort(int a[], int low, int high);
int partition(int a[], int low, int high);

int main() {
    int n;
    clock_t start, end;
    printf("Enter number of elements (n > 5000): "); scanf("%d", &n);

    int *a = (int*)malloc(n * sizeof(int));
    srand(time(NULL));
    for (int i = 0; i < n; i++) a[i] = rand() % 1000;

    printf("\nSorting %d elements using Quick Sort...\n", n);
    start = clock();
    quicksort(a, 0, n-1);
    end = clock();

    printf("\nTime taken to sort %d elements = %f seconds\n",
           n, (double)(end-start)/CLOCKS_PER_SEC);
    free(a);
    return 0;
}

void quicksort(int a[], int low, int high) {
    if (low < high) {
        int j = partition(a, low, high);
        quicksort(a, low, j-1);
        quicksort(a, j+1, high);
    }
}

int partition(int a[], int low, int high) {
    int pivot = a[low], i = low+1, j = high, temp;
    while (1) {
        while (i <= high && a[i] <= pivot) i++;
        while (a[j] > pivot) j--;
        if (i < j) { temp=a[i]; a[i]=a[j]; a[j]=temp; }
        else break;
    }
    temp = a[low]; a[low] = a[j]; a[j] = temp;
```

```
    return j;
}
```

Sample Output:

```
Sorting 10000 elements using Quick Sort...

Time taken to sort 10000 elements = 0.002500 seconds
```

LAB 11 — Merge Sort with Timing

Divide array in half, sort each half, merge back in order. Always O(n log n). Much faster than selection sort.

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace std::chrono;

void merge(int arr[], int l, int m, int r) {
    int n1 = m-l+1, n2 = r-m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l+i];
    for (int j = 0; j < n2; j++) R[j] = arr[m+1+j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r-l)/2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int sizes[] = {5000, 10000, 15000, 20000, 25000};
    srand(time(0));

    for (int s = 0; s < 5; s++) {
        int n = sizes[s];
        int *arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = rand() % 100000;

        auto start = high_resolution_clock::now();
        mergeSort(arr, 0, n-1);
        auto stop = high_resolution_clock::now();
        auto duration = duration_cast<milliseconds>(stop - start);

        cout << "Size: " << n << " Time: " << duration.count() << " ms" <<
```

```
        delete[] arr;
    }
    return 0;
}
```

Sample Output:

Size: 5000 Time: 3 ms
Size: 10000 Time: 7 ms
Size: 15000 Time: 12 ms
Size: 20000 Time: 18 ms
Size: 25000 Time: 25 ms

Python Plot:

```
import matplotlib.pyplot as plt
n    = [5000, 10000, 15000, 20000, 25000]
time = [3, 7, 12, 18, 25]
plt.plot(n, time, marker='o')
plt.xlabel("Number of Elements (n)")
plt.ylabel("Time Taken (ms)")
plt.title("Merge Sort Time Complexity")
plt.grid(True)
plt.show()
```